Coordinator: old (0.1) vs new

Measured from every Claude Code transcript on this Mac, 1–29 Oct 2026 · Coordinator now installed: **0.10.3**

Keep Coordinator. 0.1 and the new version each built the same two ADP components. 0.1's pull requests were both closed. The new version merged both, for about half the tokens, in a third of the time, with 4 messages from you instead of 23–40. What still costs time is the build-and-test gate, chains of tasks that run one after another, and visual problems only you caught. All three are fixable, and the gate problems are still open in 0.10.3.

PULL REQUESTS MERGED

2 of 2

0.1 merged 0 of 2 (#18, #19 closed)

TOKENS, SAME WORK

−54%

589M → 274M

COST INDEX

−45%

384 → 211

WALL CLOCK

34 h → 13 h

both components combined

YOUR MESSAGES

63 → 8

31 were "status?" in 0.1; 0 now

Head-to-head: the same two components

Paint thickness and service history were each built by 0.1 on 27 Oct and again by the new version in full auto on 29 Oct. The new runs started on 0.7 and picked up 0.8 and 0.9 roles mid-run, so they're labelled "new".

	PAINT THICKNESS · 0.1	PAINT THICKNESS · NEW	SERVICE HIS
Pull request	● #19 Closed	● #22 Merged	● #1
Diff	+4,102 / −310	**+3,791 / −329**	+2,7
Tokens	377M	**185M**	
Cost index	243	**139**	
Wall clock, first to last agent	23.0 h	**8.7 h**	
Wall clock with nobody active	9.4 h (41%)	**3.0 h (34%)**	3.
Agent-hours	15.6 h	**7.2 h**	
Agents spawned	64	**29**	
Agents busy at once (avg)	1.14	**1.17**	
Your messages	40	**4**	
...of which "status?"	17	**0**	
Context compactions	1	**0**	
Agents killed by a session ending	3	**0**	
Officer plan audits / task audits	none	**6 / 9**	

Cost index: tokens weighted by price, with input ×1, cache write ×1.25, cache read ×0.1 and output ×5, times Opus 5 / Sonnet 3 / Haiku 1. It reads as US dollars only if Opus input costs $5 per million tokens; use it to compare runs. A token total counts cache reads in full, which is why it overstates real cost.

What changed in the design

AREA	0.1 (RETIRED 28 OCT)	NOW (0.10.3)
Who runs the run	One Claude session leads. It spawns task leads, which spawn workers, two levels deep. Everything passes through its context.	A Python script (`coord`) schedules, merges, runs the gate, pushes and serves the live page, at zero tokens. Each agent is its own background `claude -p` session.
Parallel work	One task at a time	Independent tasks in parallel git worktrees (`max_parallel` 3)
Surviving a long run	Agents died when the session ended; the lead's context was compacted 3 times	Run state lives in `tasks.json` ; `coord wait` ; the scheduler keeps going without a session
Staying in scope	Only the lead's judgement	The hallucination officer audits the plan and each finished task against your exact words (up to 3 rounds). Plans list what's "Not in this run", and asks for another repo become follow-up runs.
Project rules	Often missed (icons, tests, comments)	`CLAUDE.md` , `AGENTS.md` and `.claude/rules` are loaded first and win over the role (0.4.0)
Tests	Written freely, including into `ADP.TestData`	Off unless you ask, enforced by a hook (0.3.0)
Comments	Heavy: you counted about 5k lines	Senior-engineer role, and the reviewer flags extra comments (0.8.0)
Seeing progress	You asked "status?" over and over	Live page with each agent's conversation, a Files tab and the waiting list. A draft PR opens at approval and is pushed after every task.
Decisions	Questions in chat, sometimes unclear	One plan approval with an answer per question. Full auto (0.6.0) decides for you, then merges and ends the run.
Pull request	Opened when you asked for one	Draft at approval. A writer agent describes it at the end, now in one short paragraph (0.10.3).
Cleanup	Manual	`terminate` deletes the run's files and each task branch once merged (0.5.1)

AREA	0.1 (RETIRED 28 OCT)	NOW (0.10.3)
Models	Opus lead, Sonnet workers (low/med/high), Sonnet or Opus reviewers	Opus by default for planner, builder, reviewer and officer; Sonnet briefer and writer; Haiku shooter. The planner can pick per task.
Platforms	macOS	macOS and Windows (0.10.0)

Where 0.1 went wrong, and whether it's fixed

#	PROBLEM IN 0.1	EVIDENCE FROM THE TRANSCRIPTS	STATUS IN 0.10.3	WHAT FIXED IT
1	The lead held the whole run in one context	Paint lead: 309 active minutes, cost 49. Compacted once in paint and twice in service history, and 4 background agents were killed when the session ended.	● Fixed	0.2: a zero-token scheduler plus separate sessions. The new driver used 35 minutes and cost 4.7.
2	Work ran one task at a time	1.10–1.14 agents busy on average; paint took 23 hours of wall clock	● Partly	Parallel worktrees exist, but the ADP plans chained their tasks, so only 1.17–1.35 agents ran at once
3	You had to keep asking for status	17 and 14 "status?"-type messages	● Fixed	Live page, `coord wait`, notifications and the waiting list. The new runs had 0 status messages.
4	Work drifted out of scope	The service-history run reworked vehicle accessories and the diff grew to about 9k lines. You asked: "why you are giving me vehicle accesories"	● Fixed	Officer audits of the plan and each task (0.2). PR #21 came in at +1,994 lines.
5	Too many comments	"way too much comment in the codes making it nearly 5k line"	● Fixed	0.8.0 senior-engineer role, with the reviewer checking
6	Project rules ignored (icons, tests in `ADP.TestData`)	You corrected both runs by hand	● Fixed	0.4.0 project instructions first; 0.3.0 tests-off hook
7	You couldn't see the changes	"feels nothing changed", and "where can I see the changes" asked 5+ times	● Mostly	Draft PR and live page Files tab (0.7.0). Before/after screenshots are still opt-in.

#	PROBLEM IN 0.1	EVIDENCE FROM THE TRANSCRIPTS	STATUS IN 0.10.3	WHAT FIXED IT
8	Unclear questions	"you didnt explain the third one to me"	● Fixed	Plan card with an answer per question (0.5.0), and full auto
9	Screenshot and perf agents took a large share of the time	Paint: 357 agent-minutes; service history: 159	● Partly	Screenshots are scripted with a Haiku shooter, but perf profiling still sits inside builder tasks (paint T4: 105 minutes)
10	Outcome	PR #19 and PR #18 closed unmerged	● Fixed	PR #22 and PR #21 merged

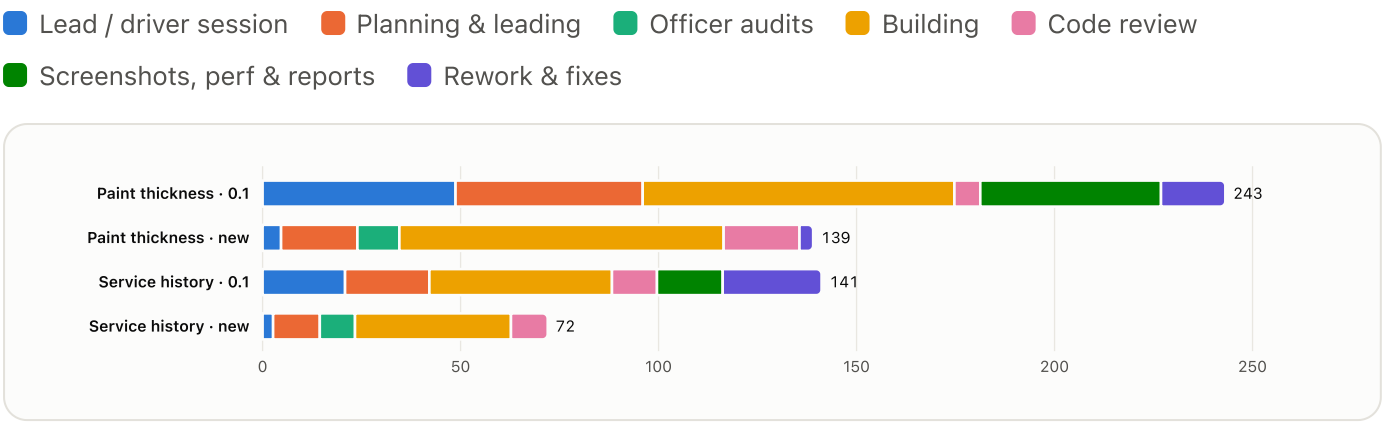
Issues seen in the new version, and their status in 0.10.3

ISSUE	WHERE IT HAPPENED	STATUS	DETAIL
Only your first prompt was acted on	skill-brain-update, 28 Oct	● Fixed 0.5.0	Additions are re-planned (P1–P5 tasks), and asks for another repo become follow-up runs
Robotic PR description	ADP #21	● Fixed 0.9–0.10.3	A writer agent; now one short paragraph
Uncommitted `.gitignore` edit diverged main	28 Oct	● Fixed 0.4.2	The ignore entry now lands through the PR
A second PR opened for the same branch	Coordinator #3 / #4	● Fixed 0.6.0	
"Idle" read as finished, so the Mac slept mid-run	27 Oct	● Fixed	`coord wait` and the run status in `tasks.json`
False gate failures	Paint run: T2-gate, T3-gate, T4-gate	● Open	Three fix agents and about 65 minutes of wall clock, and none of them changed any code. Two failures came from a stale, gitignored `src/components.d.ts` in the shared run worktree, and one from a slow spec. The fix agent only gets the last 60 lines of the log, while the real error was near the top. `integrate()` in `bin/coord` is unchanged in 0.10.3.
The full gate runs twice	Both ADP runs	● Open	Builders ran `npm run build && typecheck && test` themselves (133 minutes, at 16–18 minutes per run with 3 builders competing for CPU), then the scheduler ran it again after the merge

ISSUE	WHERE IT HAPPENED	STATUS	DETAIL
Chained plans run one task at a time	Both ADP runs	● Open	Service history ran T3 → T4 → T5 → T6 in sequence. Only 1.17–1.35 agents were busy on average, out of 3 allowed.
Visual problems only you caught, after "done"	Paint (header title, tab underline) and service history (pager in Chrome, select height)	● Open	These cost 3 extra tasks and two review rounds. Screenshots are opt-in and taken in Chromium only.
Two runs on shared components conflicted at the end	Paint T8, after #21 merged	● Open	24 minutes and cost 14. Nothing warns when two runs touch the same files.
The planner reads large context itself	Paint planner: 38 minutes, cost 19 (14% of the run)	● Partly	The briefer exists, but most of the reading of old transcripts and PR #19 stayed with Opus
Opus at high effort for every audit and review	All runs: 104 officer sessions	● Tunable	Officer, review and planning are 35% (paint) and 42% (service history) of the new runs' cost. The defaults haven't changed.
Run records are deleted on terminate	This report	● Open	No metrics survive a run, so this comparison had to be rebuilt from raw transcripts
PR fails in repos that have an `upstream` remote	kodexa-video-studio	● Open	Already on the roadmap: pass `--repo` for `origin`

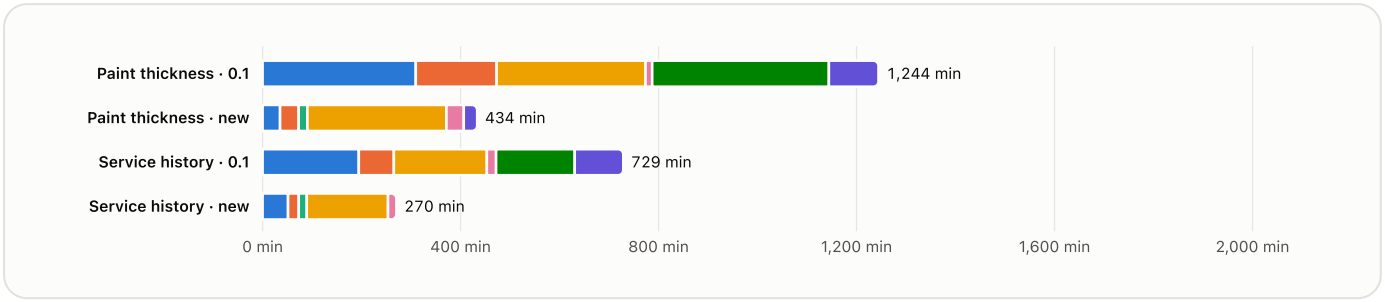
Where the time and money go

Cost by phase



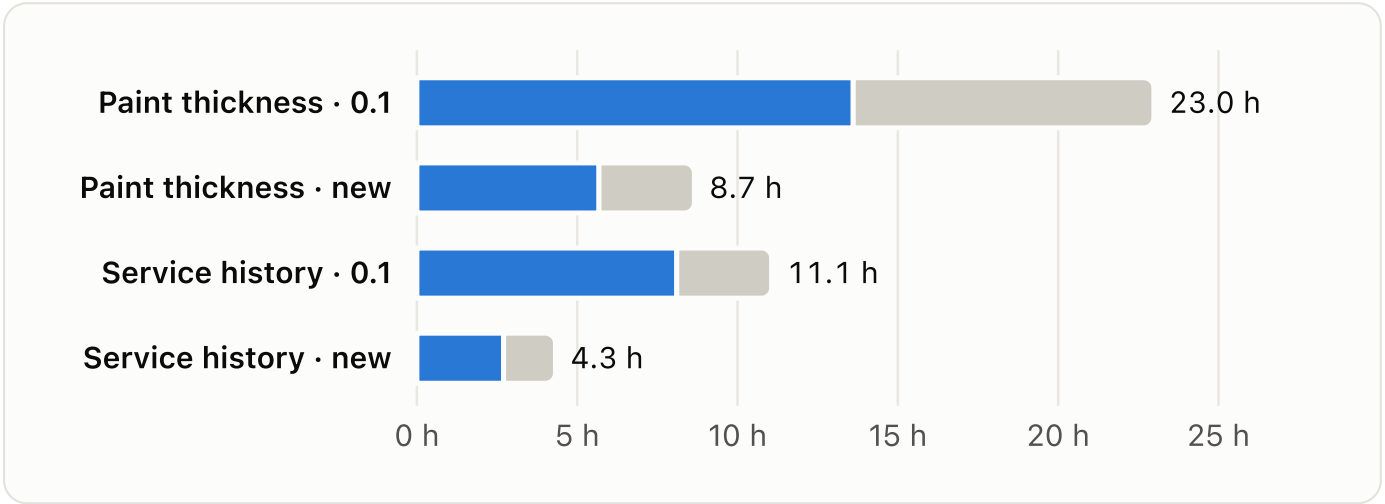
Agent-minutes by phase

The sum of every agent's active minutes. 0.1 spent most of its time on the lead session, task leads, and screenshot and perf agents. The new version spends it on building, which is the useful part.



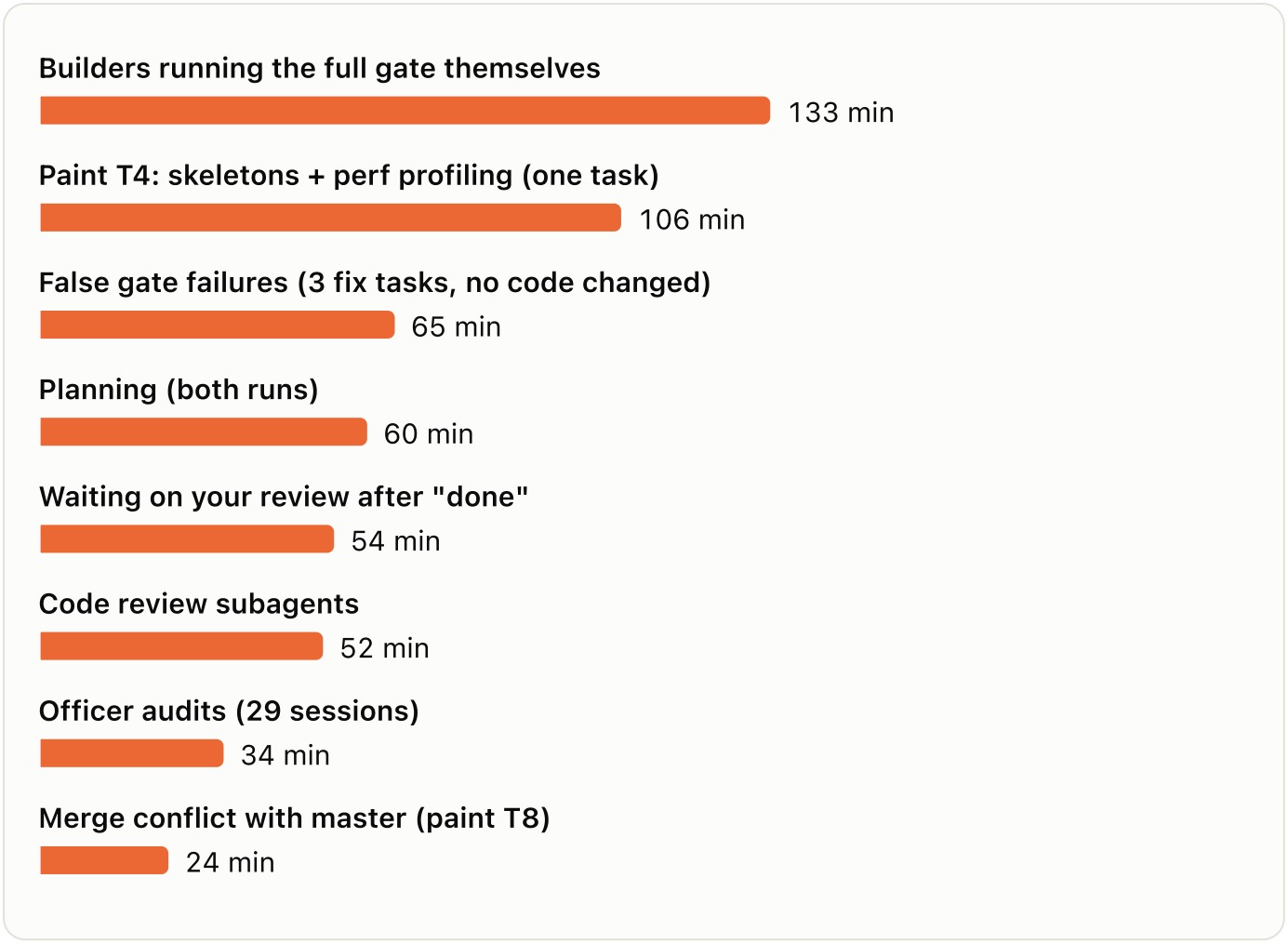
Wall clock: agents active vs nobody active

"Nobody active" covers long builds and tests, the gate, and waiting for your review.



The biggest time sinks in the new runs

Minutes in both ADP runs combined, from the transcripts



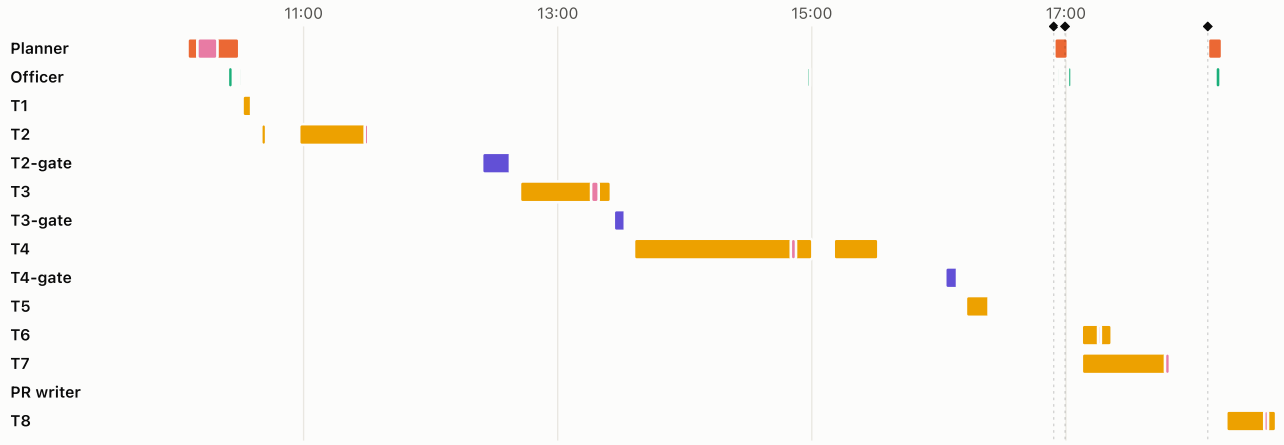
Timelines

Each bar is when an agent was actually active, coloured by phase. Diamonds mark your messages; hover or tap for detail. Each timeline fills the width: the new runs cover 4–9 hours, the old paint run 23.

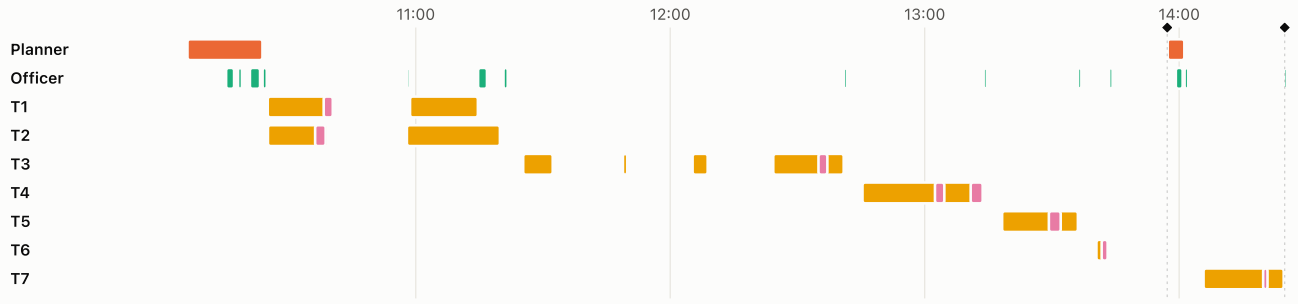
Lead / driver session · Planning & leading · Officer audits · Building · Code review

Screenshots, perf & reports · Rework & fixes · Your message

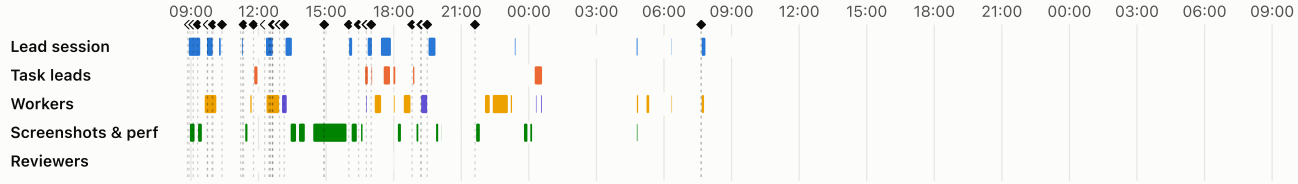
Paint thickness, new Coordinator (full auto), 29 Oct



Service history, new Coordinator (full auto), 29 Oct



Paint thickness, Coordinator 0.1, 27–28 Oct



How to make it faster and better

In order of payoff for the effort. The savings are estimates from these two runs.

1. **Make the gate trustworthy.** Re-run the gate once before creating a fix task. Delete gitignored build outputs (`git clean -fdX` under `gate_cwd`, or a `pre_gate` config) before every gate. Give the fix agent the first error block, not the last 60 lines.
   ≈65 min and 3 agents saved on the paint run
2. **Run the full gate once.** Builders run a fast check (typecheck plus related specs, e.g. `jest --findRelatedTests`), and only the scheduler runs the full gate after merging. That also frees CPU for parallel builders. most of 133 builder-minutes
3. **Plan for width.** The planner states the critical path, and the officer sends back plans that chain tasks needlessly. Split by files so redesign steps run side by side (service history ran T3→T4→T5 in sequence). Aim for 2+ agents busy instead of 1.2–1.35.
   est. 30–40% of wall clock
4. **Catch visual problems before you do.** For UI tasks, take before/after screenshots automatically in Chromium and WebKit at 390 and 1280 px, and have the reviewer check them against the design language. 4 of your 6 follow-up remarks were visual.
   a review round: ≈55 min of waiting and 3 extra tasks
5. **Right-size the models.** Put task audits and code review on Sonnet by default. Keep Opus for plan audits and L-size builds, and use medium effort for the planner on S/M runs. Officer, review and planning are 35–42% of the new runs' cost. est. 15–20% of cost
6. **Cap task size.** The planner splits any task estimated over ~45 minutes, and perf profiling becomes its own task with a time budget. Paint T4 alone took 105 minutes and cost 30, 21% of the run. shorter critical path
7. **Guard against overlapping runs.** Warn at plan time when another open run touches the same files, and merge the base branch into the run branch after each task so conflicts are small and early. paint T8: 24 min, cost 14
8. **Hand big reading to the briefer.** The planner should pass old transcripts and PRs to the Sonnet briefer rather than reading them on Opus. paint planner: cost 19 → est. 6
9. **Keep a metrics line after terminate.** Append tokens, minutes per phase, gate times and outcome to `.coordinator/metrics.jsonl`. The next comparison becomes one command, and the website can publish real numbers (an open roadmap item).
10. **Fix** `gh --repo origin` so pull requests work in repos with an `upstream` remote.

Items 1–4 alone would likely have cut the paint run from 8.7 h to about 5 h and the service run from 4.3 h to under 3 h. That estimate comes from removing the false gate tasks, the duplicate gates and one review round, and running the chained tasks in parallel.

All Coordinator runs on this Mac

The 13 real runs of the new version, plus the two 0.1 runs. Seven small test runs in temporary folders, about 8M tokens in total, are left out.

STARTED	RUN	REPO / PR	AGENTS	TOKENS	COST	WALL	BUSY AT ONCE
10-27 05:50	paint-thickness (0.1)	ADP ● #19 Closed	64	377M	243.1	23.0 h	1.14
10-27 07:39	service-history (0.1)	ADP ● #18 Closed	40	212M	141.2	11.1 h	1.10
10-28 12:59	website-docs-rebuild	Coordinator-Website ● #1 Merged	28	38M	40.1	1.4 h	1.45
10-28 14:27	skill-brain-update	Coordinator ● #1 Merged	33	65M	65.6	2.3 h	1.40
10-28 14:36	ci-deploy-fix	Coordinator-Website ● #2 Merged	6	2M	3.1	0.2 h	1.14
10-28 16:49	docs-new-features	Coordinator-Website ● #3 Merged	4	3M	3.4	0.1 h	1.13
10-28 16:58	multi-pr-merge	Coordinator ● #2 Merged	22	27M	29.0	1.1 h	1.68
10-28 18:08	docs-merge-briefer	Coordinator-Website ● #4 Merged	4	3M	3.5	0.1 h	1.19
10-28 18:22	terminate-cleanup	Coordinator ● #3 Merged	7	6M	7.2	0.3 h	1.16
10-28 18:35	terminate-cleanup-coordinator-website	Coordinator-Website ● #5 Merged	5	2M	2.5	0.1 h	1.28
10-28 18:55	coordinator-showcase-video	kodexa-video-studio ● #1 Merged	19	60M	56.3	1.7 h	1.66
10-28 19:09	coordinator-showcase-video-coordinator-w	Coordinator-Website ● #6 Merged	6	6M	6.2	1.5 h	1.05

STARTED	RUN	REPO / PR	AGENTS	TOKENS	COST	WALL	BUSY AT ONCE
10-29 07:05	fix-paint-thickness-pr	ADP ● #22 Merged	29	178M	134.4	8.7 h	1.17
10-29 07:06	service-history-redesign	ADP ● #21 Merged	22	85M	69.3	4.3 h	1.35

Method and caveats

377 sessions and 1,308 transcript files under `~/.claude/projects` were parsed. Agents were grouped into runs by the run name in their first prompt and by their worktree folder, and matched to pull requests on GitHub. Tokens are de-duplicated per API message.

Active time is time with transcript activity, where any gap over 10 minutes counts as idle. A build or test that ran longer than 10 minutes without output therefore counts as "nobody active".

Caveats. The new paint run was asked to fix PR #19, so it started from the first attempt's work and lessons. That makes the comparison fairer to 0.1 than it looks. Run logs were deleted on terminate, so gate durations are measured inside builders only. Screenshots and perf work in the new runs sit inside "Building". There are only two real ADP runs per version.